Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

Andreas Sommer, Lev Gromov, Marvin Hertweck, Pilar Keller

July 09, 2026

Task 4: Fish Farming

a) The population of fish $P(t)$ in a pond can be described by the following ODE

$$\frac{d}{dt}P(t) = rP(t)\left(1 - \frac{P(t)}{C}\right) \tag{1}$$

The constants r and C specify the growth rate and maximum capacity of the pond.

Subtask:
Use IFDIFF to solve the ODE on the time interval $I = [0, 5]$ with initial conditions $P(0) = 0.1$.
Set the constants as $r = 2$ and $C = 1$ directly in the RHS (do not pass them as parameters).
Plot the solution. What is the maximum size of the population? When does the population grow fastest?

Steps:

1. Implement the RHS in the file `rhsTask4.m` as described in eq. (1). Remember to set the constants r and C directly in the RHS.
2. Set the integration time span and initial conditions in the file `runTask4.m`. Ignore the parameters for now.
3. Use IFDIFF's `prepareDatahandleForIntegration` function to prepare the RHS for integration. Don't forget to include the integrator and its options.
4. Use IFDIFF's `solveODE` function to solve the initial value problem.
5. Use the provided function `plotPopulation(solution)` to plot your solution.

Hints:

- Check the appendix for a reminder on how to formulate and solve an ODE using IFDIFF.

Reference plot:

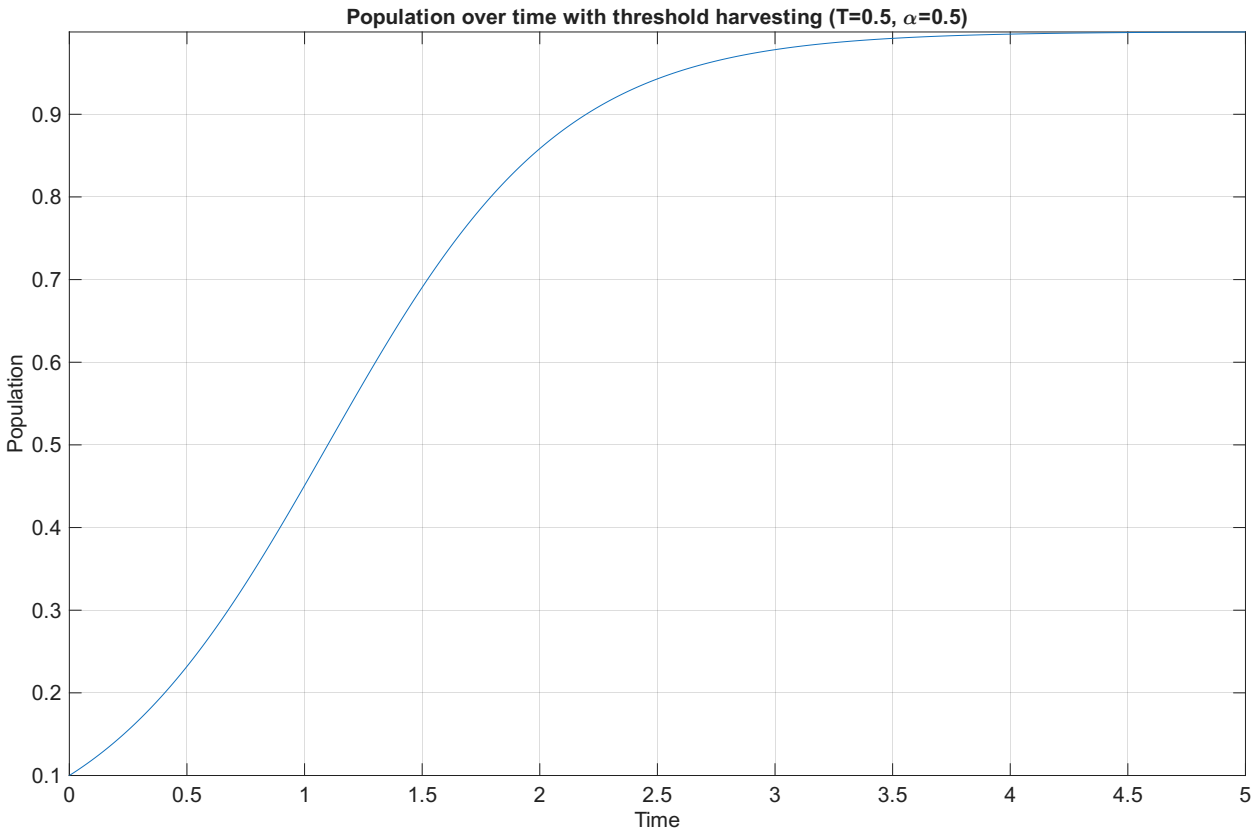

Figure 1: Population plot for subtask a)

b) The population of fish will now be harvested automatically according to the following rule:
Whenever the population $P(t)$ would exceed a threshold $T \in [0, C]$, a proportion of the current population will be extracted from the pond and added to the harvest $H(t)$ according to the harvest ratio $\alpha \in [0, 1]$.

This new system can be described by the following ODE

$$\frac{d}{dt}\begin{bmatrix} P(t) \\ H(t) \end{bmatrix} = \begin{bmatrix} rP(t)\left(1 - \frac{P(t)}{C}\right) \\ 0 \end{bmatrix} \tag{2}$$

with state jumps

$$\begin{bmatrix} P(t^+) \\ H(t^+) \end{bmatrix} = \begin{bmatrix} P(t^-) \\ H(t^-) \end{bmatrix} + \begin{bmatrix} -\alpha P(t^-) \\ \alpha P(t^-) \end{bmatrix}, \quad P(t^-) = T \tag{3}$$

Subtask:
Use IFDIFF to solve the new system for the same constants and initial conditions as in subtask a) and $H(0) = 0$.
Make sure to pass $p = (T, \alpha)$ as parameters to your RHS (do not set them directly in the RHS).
Plot the solution. What is the maximum size of the population now? Play around with different parameters.

Steps:

1. Extend your RHS from subtask a) in the file `rhsTask4.m`. Split the state vector into two components and compute the derivatives as described in eq. (2).
   Use the functions `ifdiff_jumpif(condition, direction)` and `ifdiff_update(jump)` to implement

the state jump as described in eq. (3).
Make sure that the components of your state and parameter vectors are defined in the same way as in the exercise sheet, i.e. $y = (P, H)^T$ and $p = (T, \alpha)$.

2. Add the second component to your initial conditions state vector in the file `runTask4.m`.
3. Use your code from subtask a) to solve the new system and plot the solution. Try out different values for T and α.

Hints:

- Check the appendix for a reminder on how to formulate state jumps in a RHS using IFDIFF.
- Verify that your states and parameters are defined correctly: $y = (P, H)^T$ and $p = (T, \alpha)$.
- Don't forget to include the second component of the state vector in your initial conditions.
- Make sure that your jump condition and direction match, i.e. the direction should be 1 if the jump is triggered when the condition goes from negative to positive and -1 for the other way around.

Reference plot:

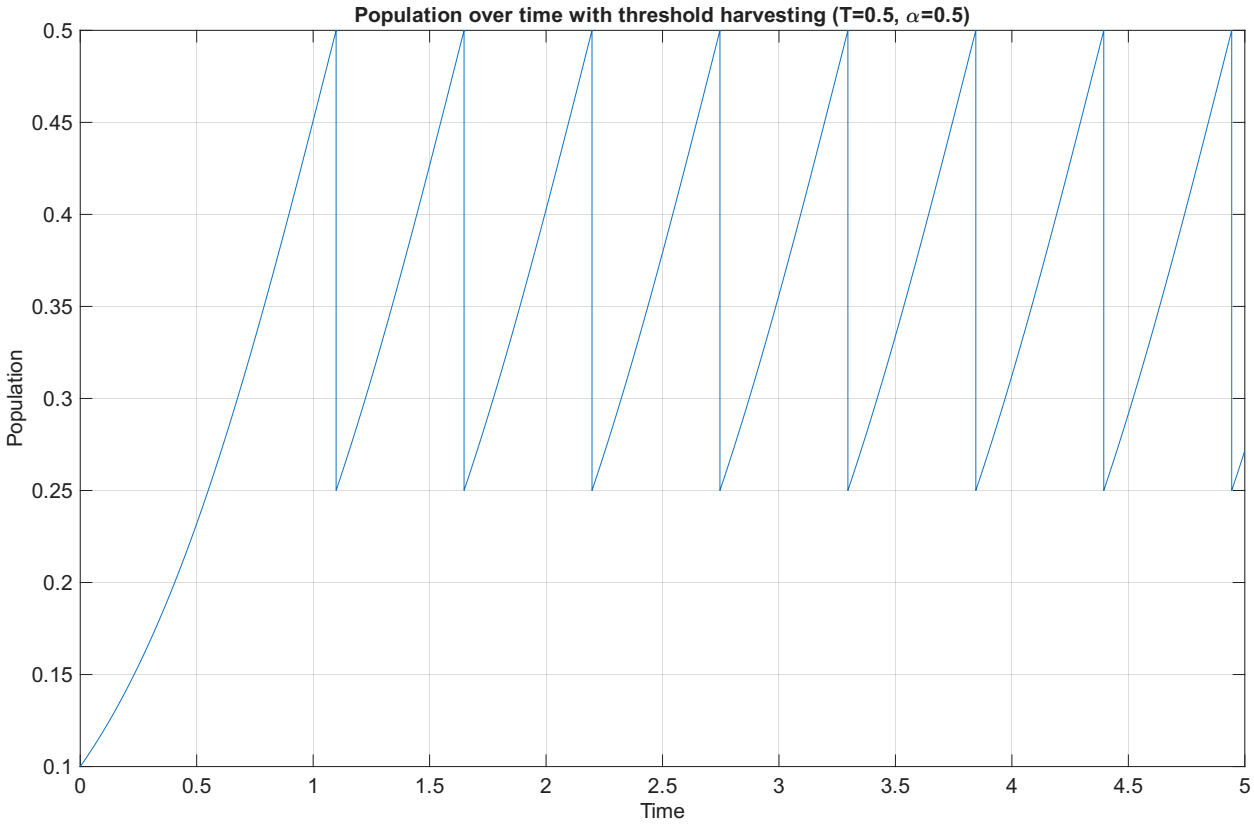

Figure 2: Population plot for subtask b)

c) We now want to examine how the solution behaves depending on the parameters T and α.

Subtask:
Discretize the parameter space $p = (T, \alpha) \in [0, C] \times [0, 1]$ as a 2D grid.
Use IFDIFF to compute a solution for each parameter in the grid and plot the results.
Explain the behavior at the grid boundaries, i.e. for $T = 0$, $T = C$ and $\alpha = 0$, $\alpha = 1$.

Steps:

1. Implement the anonymous function `solutionFunc = @(p)...` in the file `runTask4.m`, which should take a parameter vector and use IFDIFF's `solveODE` function to compute a solution for the given parameter.
2. Use the provided function `solutionGrid = solveOnGrid(solutionFunc)` to generate a solution for each parameter in the discretization grid.
3. Use the provided function `plotPopulationGrid(solutionGrid)` to plot your solutions on a grid.

Hints:

- You can define an anonymous function in MATLAB using the `@` operator followed by parantheses `()` containing the arguments of the function and finally an expression which defines the return value. Note that anonymous functions can also store copies of variables defined outside of the function and use them to compute their return value.

```
c = 5;
func = @(a, b) a + b + c;
func(2, 3) == 10 % true
```

  The anonymous function `func` defined above takes two numbers `a` and `b` and returns `a + b + 5`.

Reference plot:

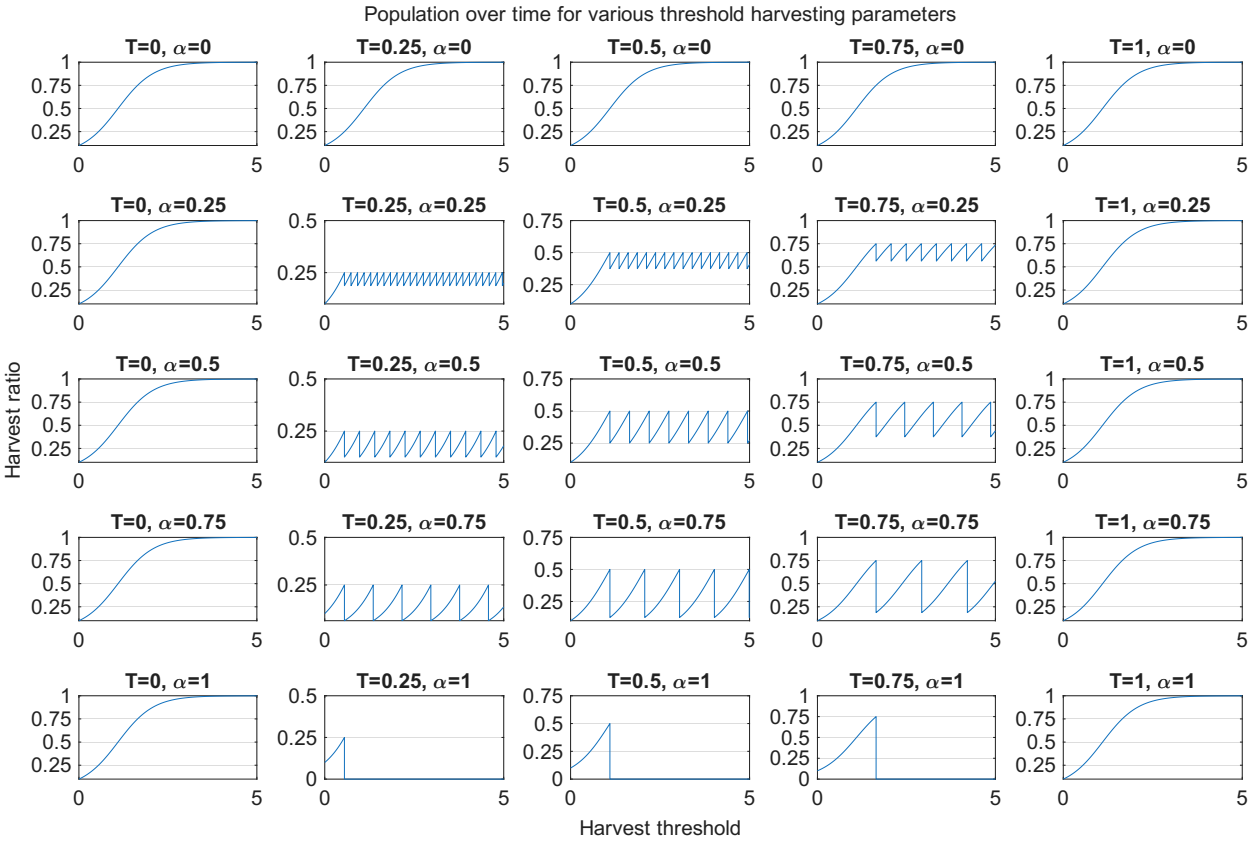

Figure 3: Population grid plot for subtask c)

d) The total amount of fish produced over the time interval $I = [0, 5]$ is given by the objective function

$$F(T, \alpha) = P(5; T, \alpha) + H(5; T, \alpha) \tag{4}$$

where $P(t; T, \alpha)$ and $H(t; T, \alpha)$ denote the solution of the system given by eq. (2) and eq. (3) with parameters T and α at time point t.

Subtask:
Compute the objective function F on a parameter grid using the solutions obtained from subtask c). Plot the result. Explain the behavior at the grid boundaries. Which combination of parameters on the grid maximizes F?

Steps:

1. Implement the anonymous function `objectiveFunc = @(solution)...` in the file `runTask4.m`, which should take a solution and return the value of the objective function F for the given solution as described in eq. (4).
2. Use the provided code `objectiveGrid = arrayfun(objectiveFunc, solutionGrid)` to apply your objective function to each solution in the grid.
3. Use the provided function `plotObjective(solutionGrid, objectiveGrid)` to plot your objective function as a 3D surface plot.

Hints:

- A solution struct (as returned by MATLAB integrators and IFDIFF) stores the timepoints at which the solution was computed in ascending order as a vector in the field `solution.x`. The field `solution.y(i, j)` stores the i-th component of the solution state vector at the timepoint `solution.x(j)`. You can access the state vector of the solution at the final timepoint in the integration time interval using `solution.y(:, end)`.

Reference plot:

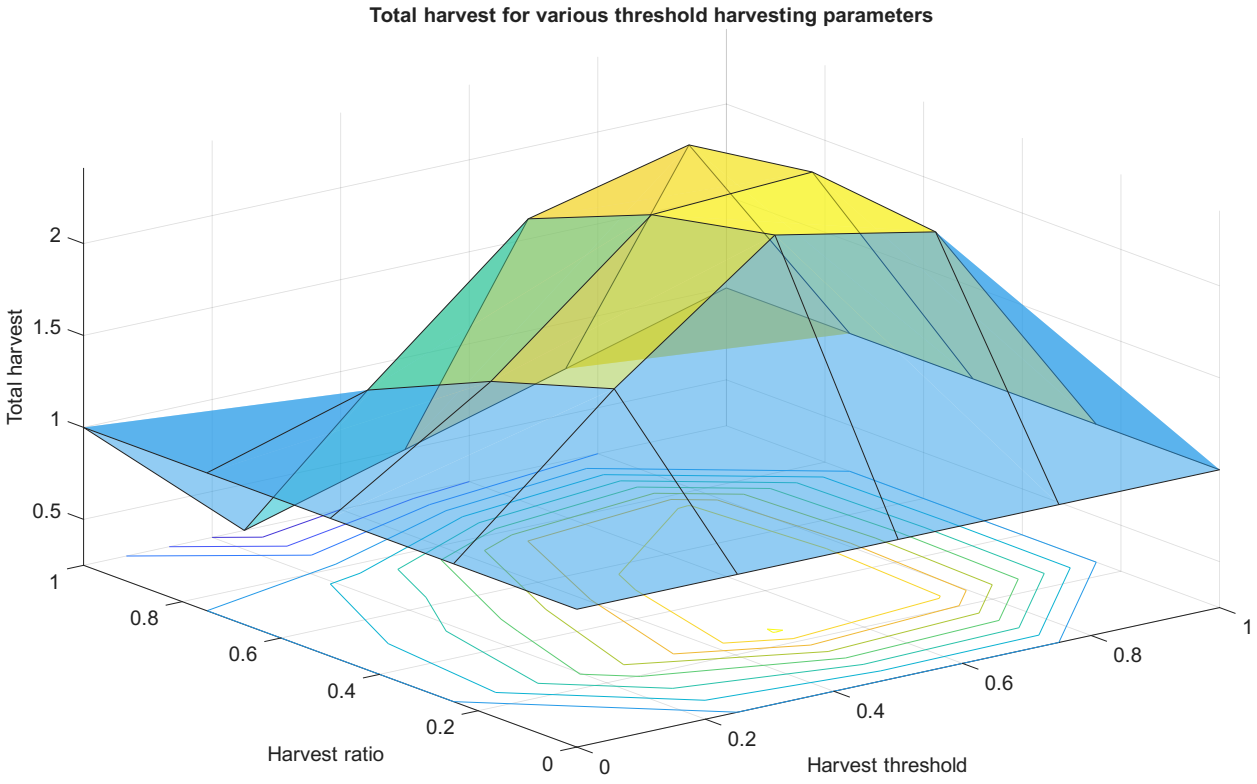

Figure 4: Objective function plot for subtask d)

e) The gradient of the objective function F is given by

$$\nabla F(T,\alpha) = \nabla P(5;T,\alpha) + \nabla H(5;T,\alpha) = (G_p)_{1,:}(5;T,\alpha) + (G_p)_{2,:}(5;T,\alpha) \tag{5}$$

where $(G_p)_{i,:}(t;T,\alpha)$ denotes the i-th row of the parameter sensitivity matrix of the system given by eq. (2) and eq. (3) with parameters T and α at time point t.

Subtask:
Compute the gradient of the objective function ∇F on a parameter grid using the solutions obtained from subtask c).
Plot the results. Where does the gradient point for the parameters which maximize F on the grid?

Steps:

1. Implement the function `g = computeGradient(datahandle, solution)` in the file `runTask4.m`, which should take a datahandle and solution and return the gradient of the objective function for the given solution as described eq. (5).
2. Use the provided code `gradientFunc = @(solution)computeGradient(datahandle, solution)` to define an anonymous function, which takes a solution and returns the gradient.
3. Use the provided code `gradientGrid = arrayfun(gradientFunc, solutionGrid, 'UniformOutput', false)` to apply your gradient function to each solution in the grid.
4. Use the provided function `plotGradient(solutionGrid, objectiveGrid, gradientGrid)` to plot your gradients as arrows in a 3D surface plot of the objective function.

Hints:

- Check the appendix for a reminder on how to compute sensitivities using IFDIFF.
- We only need the sensitivity at the integration time interval end point, which can be obtained using `solution.x(end)`.
- Since the sensitivity is only computed for one time point, you can obtain the parameter sensitivity matrix directly from the sensitivity output struct using `sens.Gp`.
- Since we only need the parameter sensitivities and not the initial value sensitivities, you can pass the optional flag `generateSensitivityFunction(..., 'calcGy', false)` to prevent unnecessary computations.

Reference plot:

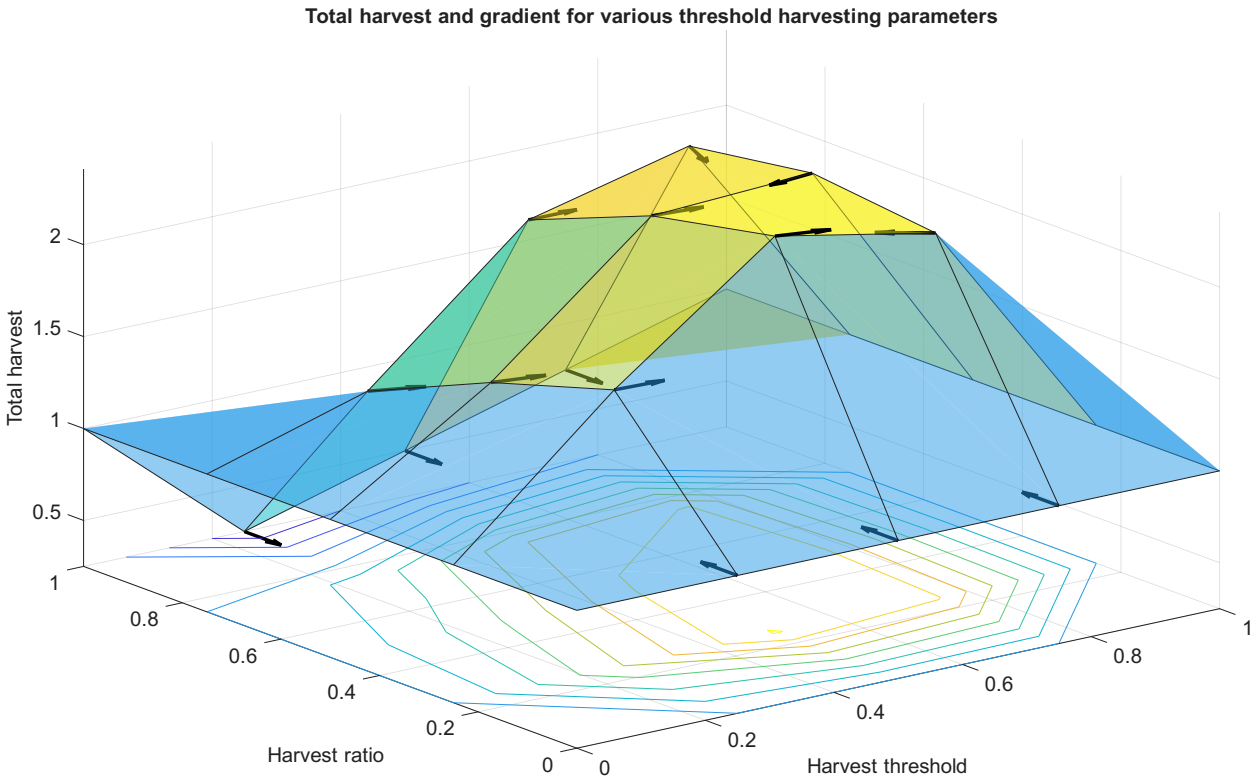

Figure 5: Gradient plot for subtask e)

f) After computing the gradients, we can now optimize the parameters T and α in order to maximize the total amount of fish produced as described by the objective function F. However, we will additionaly impose a lower bound constraint on the harvest ratio, namely $\alpha \geq 0.2$, because the number of state jumps becomes arbitrarily large as α goes towards zero, which is intractable to handle numerically.

The optimal parameters T^* and α^* are then defined via

$$F(T^*, \alpha^*) = \max_{T \in [0,C], \alpha \in [0.2,1]} F(T, \alpha) \tag{6}$$

Subtask:
Use MATLAB's optimizer `fmincon` to compute the optimal parameters T^* and α^* which maximize the objective function F as described in eq. (6).
Use IFDIFF to compute the solution for the optimal parameters and plot the result. What are the maximum and minimum values of the population after the first harvest? Which point lies exactly between the maximum and minimum values? Can you explain intuitively why the midpoint has to lie where it does for the optimal parameters? Think back to the solution without state jumps. When is the population growth maximized?

Steps:

1. Implement the function `[f, g] = objWithGrad(p, solutionFunc, objectiveFunc, gradientFunc)` in the file `runTask4.m`. The function should take a parameter vector and the solution, objective and gradient functions that you have implemented in the previous subtasks and return the *negative* objective `f` and gradient `g` for the given parameters. It is crucial that the objective and gradient are negated because MATLAB optimizers can only handle minimization problems, so we need to flip the signs of our objective and gradient to transform our maximization problem into an equivalent minimization problem.

2. Use the provided code `optimizationFunc = @(p)objWithGrad(p, solutionFunc, objectiveFunc, gradientFunc)` to define an anonymous function which takes a parameter vector and returns the negated objective and gradient.
3. Use the provided function `[pOpt, objOpt] = optimizeParameters(optimizationFunc)` to setup the optimization problem and compute the optimal parameters and objective value using `fmincon`.
4. Compute the solution for the optimal parameters using the function `solutionFunc` from subtask c).
5. Use the provided function `plotPopulation(solutionOpt)` to plot the solution for the optimal parameters.

Hints:

- Remember that `solutionFunc` takes a parameter and returns a solution, whereas `objectiveFunc` and `gradientFunc` take a solution and return the objective and gradient.
- Don't forget to negate your objective and gradient in the function `objWithGrad`.

Reference plot:

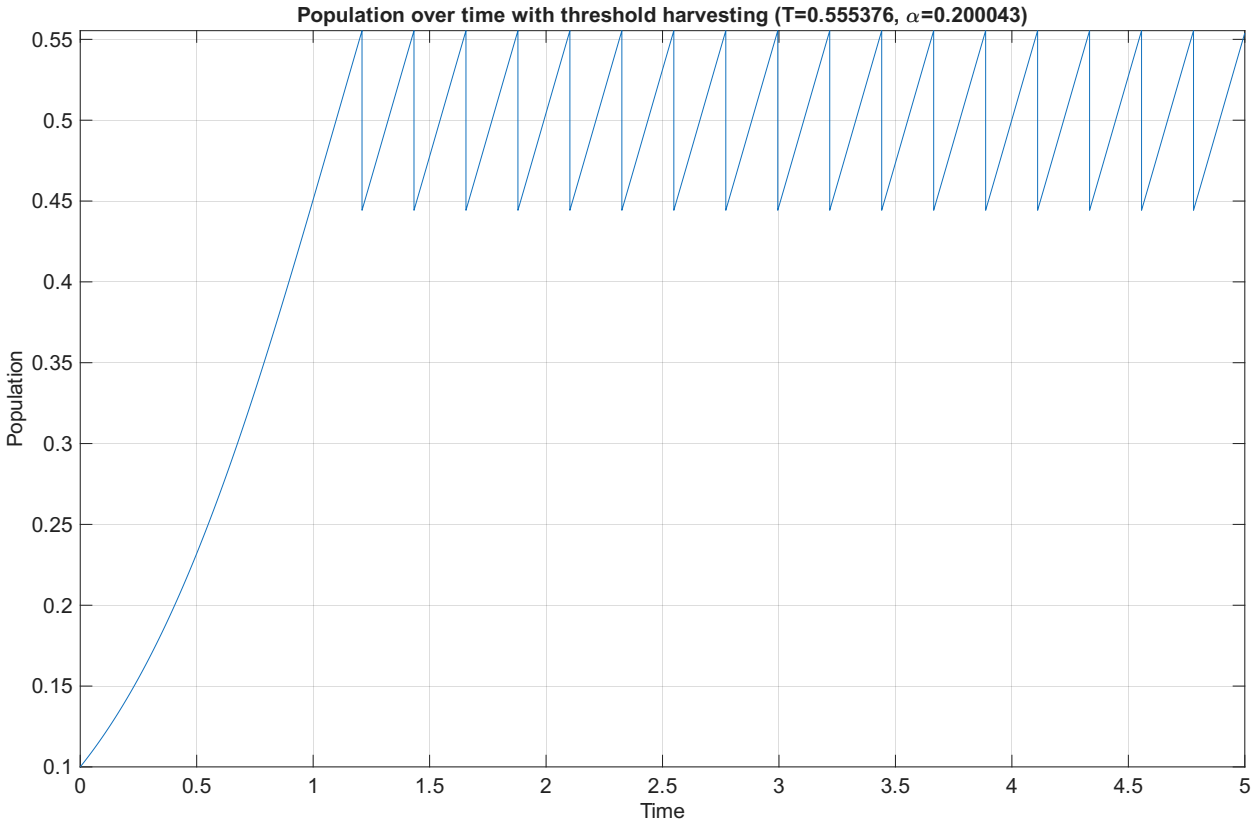

Figure 6: Optimized population plot for subtask f)